

Flight Software Systems
(ENAE 380 - Sec. 0106)

Final Project Design PDF

Clara Shoppa

December 15, 2025

i. Introduction

For my final project I developed a desktop workout creation and tracking application using Tkinter in Python. I chose this project because weight-lifting and fitness are major passions in my life, and I wanted a tool that reflects the flexibility of how real people train. In researching existing workout-tracking apps, I found that many popular options, such as JEFIT¹, SHRED², and Hevy³, limit the user's ability to create custom exercises and plans unless they pay for a subscription. These apps also require users to create accounts, provide personal information like height and weight, and multiple goal-setting questions before they can even access the home screen. This makes starting a simple workout unnecessarily slow and restrictive, and creates additional fears in data-privacy, lacks function offline, and has a heavy and slower UI due to the complexity.

My application, LiftUp, is designed to address these issues by providing a simple, free, and fully customizable workout-tracking experience. Users can create unlimited exercises, build personalized workout plans, and track sets in real time without accounts, paywalls, or onboarding requirements.

Before beginning on the application I knew that I wanted to use an object-oriented Tkinter architecture to create the application. Choosing Tkinter as the development framework was intentional as Tkinter is the default GUI Toolkit for Python, which allows for most compatibility across platforms, and requires no installation. While Tkinter is not common in commercial applications, it is the foundation for many modern GUI applications and is highly accessible to learn and understand.

I conducted a preliminary study of Tkinter fundamentals. I followed several online tutorials to understand pack and grid layout managers, creating and styling widgets, and how to use class to modularize GUI pages.⁴ I then searched for already existing workout planners and trackers on the internet, and found some examples online. This⁵ example is not quite functional, but it gives me a guide on how to use a dictionary to store plan information in a dictionary through the use of binding functions with buttons. This⁶ example implements the import datetime

¹ <https://www.jefit.com/>

² <https://www.shred.app/>

³ <https://www.hevyapp.com/>

⁴ <https://www.pythontutorial.net/tkinter/tkinter-object-oriented-frame/>

⁵ <https://ideone.com/ittn9n>

⁶ <https://medium.com/dataregressed/weightlifting-tracker-gui-with-tkinter-and-sqlite3-f34a33f3ce8e>

to track the time of the workout. To expand these existing projects, I researched how to organize and use multi-frame views⁷ and how to handle user interactions within a view, and how it updates the or switches between different frames as needed. This allows them to manage complexity in larger applications, and increase modularity.

I also did research into how I could store my data. Many projects use SQL or JSON. I chose to research and use JSON data formatting because it uses simple syntax, very similar to dictionaries and lists, and has a hierarchical and nested structure that can easily be used for nesting exercises, and set information, within one workout plan. It can also work with datetime⁸ import to save and retrieve time data.

Functionally, LiftUp allows for users to create an unlimited number of workout plans, each containing custom exercises and sets. During a workout session, the application guides the user through the exercises set-by-set while tracking the number of completed sets, remaining sets, and user-entered performance data such as reps and weight. All created plans and completed sets are saved into a structured log that can be analyzed through the dashboard within the application, providing a clear and interactive training experience similar to commercial fitness apps but without the associated restrictions.

7

<https://nazmul-ahsan.medium.com/how-to-organize-multi-frame-tkinter-application-with-mvc-pattern-79247efbb02b>
<https://www.youtube.com/watch?v=3yeRcxkth0I>

8

https://medium.com/@ryan_forrester_/save-and-load-json-files-in-python-a-complete-guide-49a5760b8b49

ii. Design Process

The design of LiftUp evolved through several iterations, and much of the development process focused on structuring the application into clear, modular page classes. Early in the project, I realized that placing all logic into a single Tkinter window became quickly unmanageable. To avoid this, I adopted a multi-page architecture, where each major feature, such as creating a plan, or tracking a workout– was implemented as an independent class inheriting from a `ttk.Frame`. This decision significantly improved clarity, as every page became responsible for its own layout, widgets, and user interactions.

I created the application in the class `LiftUp` by defining an App class named ‘`LiftUp`’ that inherits from the `Tk` class, as well as adding the `if __name__ == "__main__":` block to create an instance of the App class and setting it to a basic container in the app, and executing it with invoking the main loop of the root window with `app.mainloop()`. By following the techniques I learned in the Tkinter tutorials, I created a top frame within the `LiftUp` app to act as a permanent header to the app. I used the grid layout to organize, and played around with hexadecimal colors for styling. I created a button that will be tested to switch between pages in the container frame underneath this frame, but for now the command for this button was set as none.

The next step was to define the format for the completed workout logs for the dashboard. The default page I want displayed when the page to open up is a dashboard for general tracking information and displaying recent workouts completed. To do this, it would have to pull the workout log information from a data file. I created the data file ‘`WorkoutLogs.json`’ and considered how I want the data to be organized. The workout log will have a main list of each total workout completed. Each workout plan will have its main general key-value pairs, such as a “`plan_name`” to easily call the workout log, a “`plan_index`” to keep track of where it is in the log, a data, start and end time for data tracking information for the user, and its own dictionary for its “`exercises`” to hold its key-value pairs for the reps, weight, and a boolean for if it was completed or not to allow user to skip a set and track the workouts. I filled the file with this mock data for testing out my `DashboardPage` class functionality:

```

1  [
2  {
3    "plan_name": "Upper Body Strength",
4    "plan_index": 0,
5    "date": "2025-02-01",
6    "start_time": "2025-02-01T14:12:33.582913",
7    "end_time": "2025-02-01T14:45:10.112399",
8    "exercises": [
9      {
10       "name": "Bench Press",
11       "sets": [
12         {
13           "target_reps": 8,
14           "target_weight": 135,
15           "completed": true,
16           "reps_done": 8,
17           "weight_done": 135
18         },
19         {
20           "target_reps": 8,
21           "target_weight": 135,
22           "completed": false,
23           "reps_done": null,
24           "weight_done": null
25         }
26       ]
27     },
28     {
29       "name": "Overhead Press",
30       "sets": [
31         {
32           "target_reps": 6,
33           "target_weight": 95,
34           "completed": true,
35           "reps_done": 6,
36           "weight_done": 95
37         }
38       ]
39     }
40   ]
41 }
42 ]

```

Fig 1. Structure of WorkoutLog.json file

Inside the LiftUp app, I loaded the data files into public variables named “workout_logs” and “workout_plans” so that I can easily write and call data internally between LiftUp’s child classes. To load json files, I created a helper function called “**load_json**”, as well as a “**save_json**” function to load data files for later use within the app development. I based these functions on code I found from my research on json files.

The **DashboardPage** class will inherit from both the `ttk.Frame`, so it will be a frame not an app, and the LiftUp app, to take advantage of public and private variables between classes. Within LiftUp, I created a container frame that will hold the DashBoardPage widgets, and initialized the dashboard using the container frame, and stores it in a dictionary called “pages”. I then defined a function **show_dashboard()** that uses **.tkraise()** to raise the object called “DashboardPage” that was already initialized outside this function. Then I attached the `show_dashboard()` function to the dashboard button. Within the DashboardPage class, I created a basic label, so I can test if the button opens the frame, and it did. With this working, I continued with developing the DashboardPage class and its functions.

The DashboardPage will show the user their total workout’s, workout’s done this week, and total sets completed, and the average duration of each workout. To show each of these variables, they will be set using a refresh function, and displayed within their own frame in the main initializer. The DashboardPage will also have a frame underneath to show information about recent logs. This frame will be created in the DashboardPage initialization, but data will be filled using the refresh function as well.

DashboardPage's **refresh()** function uses the workout logs loaded from the main App, and retrieves and computes the variables required to fill the variables in the DashboardPage. I first focused on the four variables that will be shown at the top of the DashboardPage's frame.

1. The total number of workouts: **total_workouts_var**. This is computed from finding the length of the workout_logs list, as each entry in this list represents a workout.
2. The total number of workouts completed in this current week: **this_week_var**. This was very challenging to achieve, and I used several resources ⁹on using datetime import to figure out. First, we need to find when this week started, which is specifically defined as the date of the most recent Monday. I used datetime.now() to find the current time, and then then datetime(now,year, now.month, now.day) to ignore the specific time of day, so we can't start at 00:00 on Monday, instead of what time of day it currently is. Then to make the index subtractable to the datetime() value, time delta() is required. To obtain the time stamps of the data, I created a helper function parse_dt¹⁰() to return obtain the data and time of data using, or None if there is no data. Then in refresh(), it goes through each log, and adds to this_week_var if the workout's end time is greater than the start of the week.
3. The total sets complete: **sets_completed_var**. I used nested for loops, as for each log contains its "exercises", and within each exercise it contains its "sets", and if the sets "completed" boolean is true, then it add ones to the variable.
4. The average duration of a workout: **avg_duration_var**. I used the for loop used to get sets_completed_var, and got the start and end time of each workout log, allowing us to get the total seconds, and we can divide by 60 to get the duration of each workout in minutes. This is appended to a list, and after the for loop, the average was calculated to be set as the avg_duration_var.

I then added the dashboard's refresh function to the show_dashboard function in the App's class to allow for the data to be loaded when showing the page. I tested the refresh function, and it worked successfully after a few syntax tweaks.

⁹ <https://docs.python.org/3/library/datetime.html>
<https://stackoverflow.com/questions/42036846/a-simpler-way-of-finding-the-start-of-the-week-for-any-start-day/42037331#42037331>

¹⁰ <https://stackoverflow.com/questions/127803/how-do-i-parse-an-iso-8601-formatted-date-and-time>

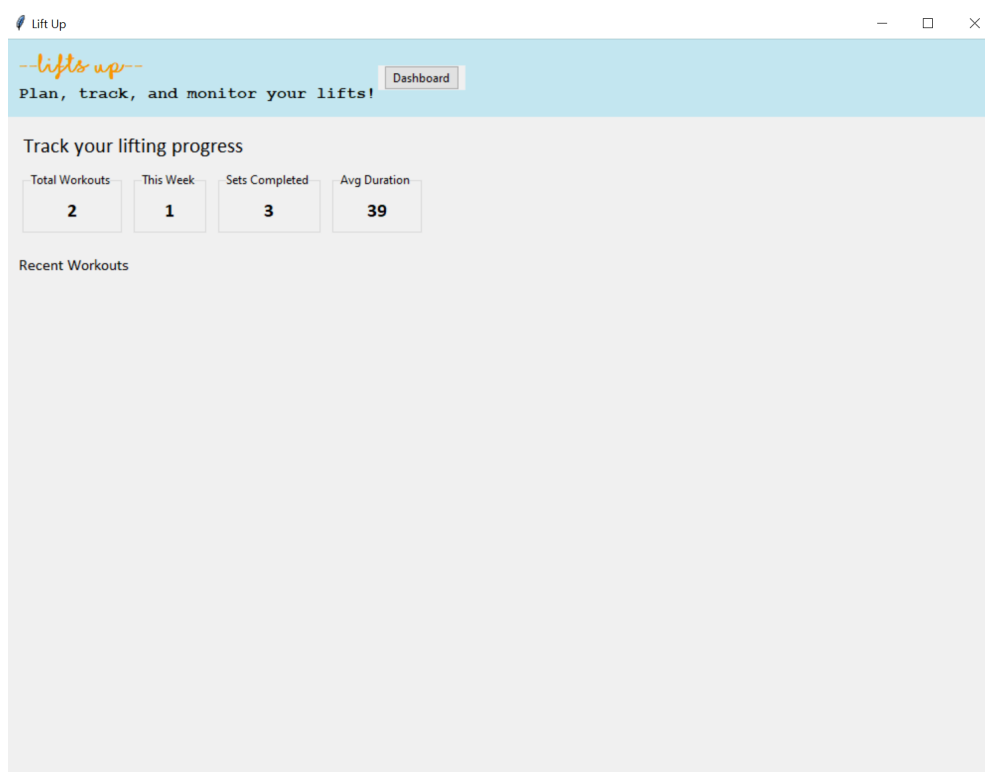


Fig 2. Data formed from WorkoutLogs to create Dashboard Page.

The next part of the DashboardPage class is the frame for holding recent workouts. Within the refresh function, I created a sorted list of recently completed workouts, and pulled out basic information pertaining to the workout such as the date, number of exercises, and sets within each workout. I displayed this information by using a for loop outside of refresh for the items in a recent workout that creates a frame, attached to a custom scrolling frame object, for each workout with proper labels. I created a custom scrolling frame, **ScrollableFrame(ttk.Frame)** to utilize the scrollbar widget, and allow for more information to be neatly displayed in one frame. I used ****kwargs** as an input for the frame to allow for flexibility when initializing the frame.

When testing, it correctly displayed the information I wanted within the frames for each recent workout. However everytime the Dashboard button was pressed, it created a new copy underneath, and the scroll bar was not attached to the canvas correctly, as it was able to scroll up into empty space. I fixed the duplicating of frames by adding a for loop to destroy each widget inside the scrollable frame before widgets are created in the scrollable frame. To fix the scrollbar, I binded a custom function ¹¹when the canvas is reconfigured. When the canvas is reconfigured, the canvas and content size information is gathered to prevent scrolling when the content is shorter than the view.

¹¹

<https://stackoverflow.com/questions/16188420/tkinter-scrollbar-for-frame>

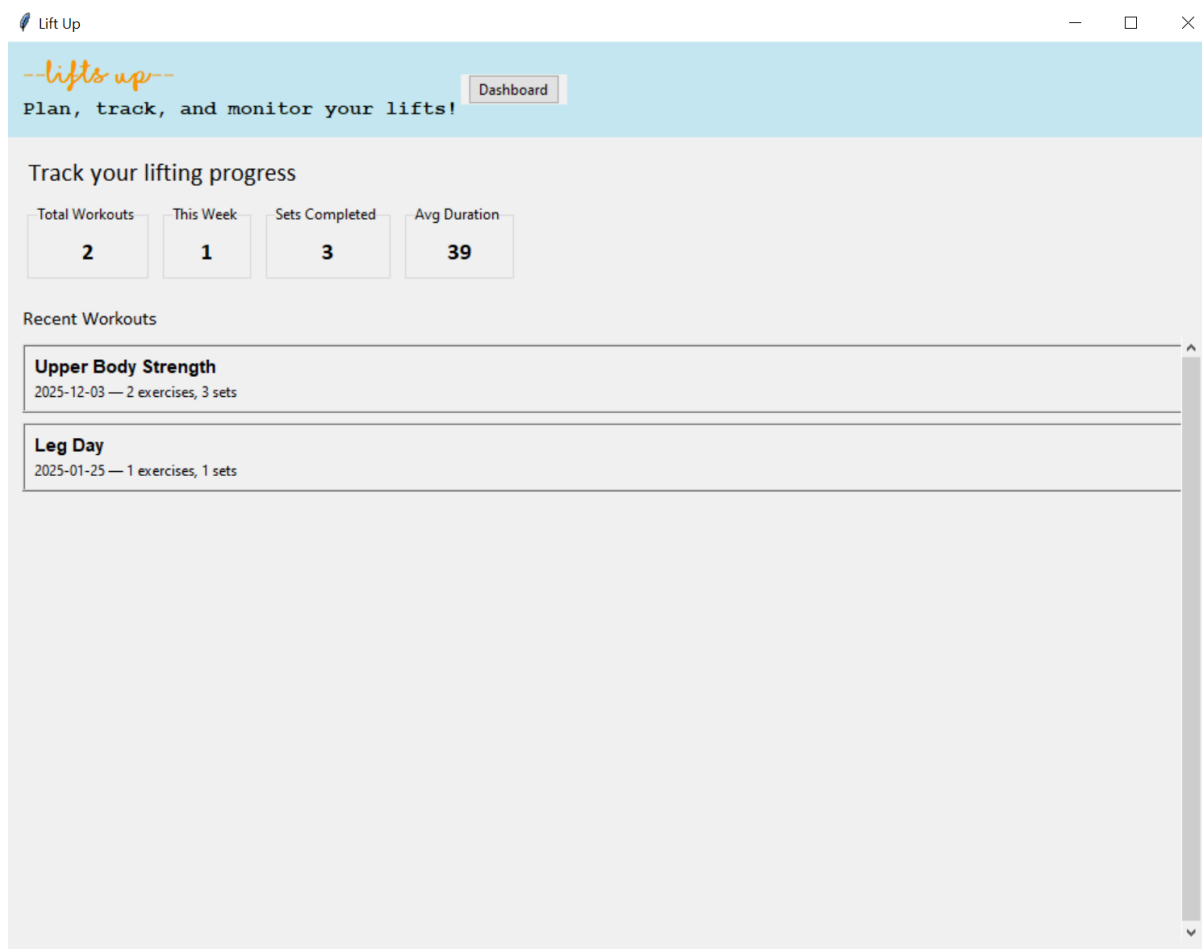


Fig 3. Recent workout logs populated and displayed in a scrolling frame in Dashboard Page.

With the DashboardPage complete for now, I moved onto the **PlansPage**. This page class will have a top frame with a header and a button to create a new plan, which will switch the space below from holding a list of already created workout plans using a scrollable frame, and a frame for the user to create a custom new plan with the use of buttons and entry widgets.

The initialization of the PlansPage mirrored the initialization of the dashboard, and created a function **show_plans** in LiftUp to bind the new “Plans” page button in the LiftUp app to raise and apply the PlansPage’s refresh function. The PlansPage itself will have a button to show a frame to create a new plan, but for now I focused on displaying already existing plans. I created mock-data in the workoutPlans.data to test this. To get the data, it followed the same log as getting the data in workoutLogs in the DashboardPage’s refresh by using the PlansPage refresh function to assign data to variables, and then fill the frame with those variables in the **function show_list_view**, using a very similar format within my custom ScrollableFrame class.

To create a new plan, the button labeled “+ New Plan” has the command of the function **show_new_plan_frame()**, which switches the frame on the bottom of the screen from a list view. This function requires another function, **build_new_plan_frame()** is called in the

initialization of the PlansPage class, to actually build the frame with widgets. It creates a tk.entry for entering a new “Plan Name” and stores it as a public variable **plan_name_var()**, and also creates another scrollable frame to view the exercises, that each will have its own frames. There is also a save and cancel button within this scrollable frame that will be attached to custom commands to save and cancel the new workout plan. Each exercise frame will be held within a dictionary “exercise_frames”. Within each frame for each exercise, it will have entries for entering the exercise name which will be saved **ex_name_var**, another frame for adding set information, and a button to add a new exercise. For creating these exercise frames, I created the function **add_exercise_frame()** that is called in show_new_plan_frame to initially create an add_exercise_frame when a new plan is made, and also the command in the “add exercise” button in build_new_plan_frame(). Within add_exercise_frame(), the function add_set() is defined to similarly add sets within each exercise. Within each set frame, there are entries for the reps, and weight numbers. These entries are appended to the dictionary exercises_frames.

The screenshot shows a window titled "Create New Plan". At the top, there is a label "Plan Name" followed by a text entry field. Below this is a section titled "Exercises" which contains a scrollable frame. Inside the scrollable frame, there are three individual exercise frames, each labeled "Exercise1". Each exercise frame contains a label "Exercise Name:" followed by a text entry field. The first exercise frame has the text "Hi" in its entry field, the second has "Test", and the third is empty. At the bottom of the scrollable frame, there is a button labeled "+ Add Exercise".

Fig 4. Nesting add_exercise_frame within active_workout_frame for PlansPage.

Next I added a button to “Remove Exercises” with the exercise frame so that an exercise can be deleted, and created the function **remove_exercise()**. This button destroys the current frame with “frame.destroy”, scans through the exercise_frames dictionary to remove the plan matching the current index, and then re-labels the remaining exercises frames starting from work. This successfully worked.

The figure consists of two side-by-side screenshots of a web form titled "Exercises".

Left Screenshot:

- Exercise 1:** The "Exercise Name:" field contains "This will be deleted". Below it, "Set 1" has input fields for "10" reps and "10" lbs.
- Exercise 2:** The "Exercise Name:" field contains "This will stay". Below it, "Set 1" has input fields for "20" reps and "20" lbs.
- Buttons: "+ Add Set" and "Remove Exercise" are present for each exercise.

Right Screenshot:

- Exercise 1:** The "Exercise Name:" field contains "This will stay". Below it, "Set 1" has input fields for "20" reps and "20" lbs.
- Buttons: "+ Add Set" and "Remove Exercise" are present for each exercise.

Fig 5. Functionality of remove_exercise() function.

I then created a refresh() function to refresh the PlansPage by showing the list view automatically, and destroy any current build_new_plane_frame's. This refresh button can be used for the command of the "Cancel" button when creating a new plan, and is also used in the LiftUp App's show the plans page.

To finish the PlansPage class, I created the functions for the buttons I left without commands.

1. **save_plan()** function saves the plan name, exercise name, and set information from the private variables to the public storage, and json files. It uses the variables created in build_new_plan, such as plan_name_var, and exercise_frames dictionary, to append to the app's internal list of plans, and uses the helper function save_json to save to the external json file. This function checks that each entry is a valid integer before saving. If the entry is not valid, it will show an error message box to inform the user.
2. **delete_plan()** function is used for "deleting" an already made plan that is shown in the list view. This function simply takes in the index of the plan from the iteration in show_list_view, then deletes the plan in the App's internal storage, workout_plans, and calls save_json to save the new plan, and finally refreshes. Before deleting the plan, it shows an "askyesorno" message box if the user is sure to delete the plan. A lambda function was implemented as a solution to attaching a function with an argument to a button command without having it run by default. If we used the argument in this function, but did not use a lambda function within the button's command, it would automatically try to delete every plan.
3. **edit_plan()** function was created to allow the user to edit an already existing plan, and show_new_plan_frame(), add_exercise_frame(), and save_plan() functions were

modified to allow for this. A new variable, “edit_index” is set to None when we are not editing, and sets the index of the plan being edited when the “edit plan” button is clicked. It then adds this as a parameter to show_new_plans, and prefills the show_new_plan and add_exercise_frame with the plan information. For example, the ex_name_var is defined to be the “name” from the plan with the corresponding index to the edit index if there is a prefill, otherwise it is defined to be empty. This prefill is applied similarly to the reps and weight variables.

4. **cancel_edit()** function was created to account for resetting the edit_index back to None. Without this function, it would keep this index stored and would show an “editing plans” page instead of “create new plans” page when the “create new plan” button is pressed.

The third and final page in the LiftUp app is the **TrackPage**. This page allows the user to select a workout from a dropdown of already created workouts, and then to start, follow, and track the workout plan. For a workout, it goes through each exercise, and each set for the workout, and displays the original rep and set information saved in the plan as the “targets”, and entries for the user to enter in the amount of weights and rep they actually completed. This data is then saved internally to the App’s workout logs, and the workout logs json files.

I created the combobox, plan_combo to be created during the initialization of TrackPage, but filled with values from the TrackPage’s refresh. The combo box has a function **plan_selected()** binded to the selection action. When a plan is selected from the combobox, a frame underneath displays the basic workout information, such as the exercise names, in one string. The plan_selected() function gathers the basic workout information variables into one string as a public variable to be accessed outside of the plan_selected() function.

The active frame that is hidden until the workout is started will be controlled by the public variable current_workout. This variable is initially set to none within the LiftUp app, but then is initialized with the plan selected information as a dictionary in the new function **start_workout()**. This function is activated when the “Start Workout” button is clicked. The function start_workout() ensures that the current plan is selected by checking the index of the selection is greater than zero. It creates a new dictionary, called plan directly from the current_workout variable, and is filled with the information from that plan, but adds to the nested “sets” dictionary, and holds a new term “progress.” This new term allows the program to keep track of where we are in the workout by holding our current exercise and set index. This index is set to zero, and is updated throughout the workout in the next_set() function. The “sets” term is modified to hold the original reps integer under “target_reps”, and the original weight integer under “target_weight”, while holding empty values for “reps_done” and “weight_done”. This allows us to hold both the targeted data and what was actually completed, but not override the original workout plans. Finally, start_workout() calls the function show_active_workout() to construct and fill the frame.

The function **show_active_workout()** fills the active workout frame with the plan name, and a nested frame to display the current exercise’s information that is retrieved from start_workout(). Within the current exercise frame, it displays the exercise names, our targets, progress in the

workout, and it holds entry boxes for the user to enter in their completed set numbers. It also has a button to continue the workout by either skipping the set, or completing the set. There is an essential check in this function to see if the workout is complete. It does this by checking if our current exercise index is greater or equal to the length of the exercise lists.

To move onto the next set or exercise, regardless if the last set was completed or skipped, it needs to find where the next exercise or set is, and advance our current position to that position, and update the active workout frame to match this placement. This is achieved in the **next_set()** function. This function gets our current index from the “progress” term in our current_workout dictionary, and then checks the exercise index first. We must check the exercise index before checking the set because we may already be at the end of the workout, then it returns nothing, and the workout would finish using the check within show_active_workout(). If the exercise index is valid, then we must check if we are at the end of the sets list and move onto the next exercise, or to go on to the next set. This function only updates the position, the “progress” of the workout, but does not change the “set” information, so the set is left marked false under the “completed” term. This function works directly as the command for the “skip set” button.

The function **complete_set** completes the next_set() function by updating the “saved_sets” term in the current_workout dictionary. It updates the “completed” term to True, and sets the reps and weight done as the input from the entries by the user. It then uses the next_set() function to update the positions and current workout frame.

The **exit_workout()** function was easily made by implementing another “askyesno” messagebox to ensure that the user is sure to press the “Exit” button, and to leave the active workout frame. If the user presses yes, then it resets the app’s current_workout state back to None, and refreshes the TrackPage.

Finally, the **finish_and_save()** function was created to end the workout, and to save to both internal and external memory. It creates a new dictionary from the current_workout dictionary in the proper format to save with. It then appends this new log entry to the app’s internal permanent memory within the variable workout_logs, and uses the save_json data helper to save to the external json file. Then to finish the workout, like exit_workout(), it sets the app’s current_workout status to none and refreshes the page.

--lifts up--
Plan, track, and monitor your lifts! Dashboard Plans Track

Start Your Workout

Choose a plan and start your guided workout session:

Select Workout Plan: Legs

Plan Info
4 exercises • 12 sets
Exercises: Hip Thrust, Deadlift, Hamstring-Curl, Unnamed Exercise

Start Workout

Exit **Legs**

Hip Thrust
Set 1 of 3
Target: 15 reps @ 150 lbs

Reps Completed Weight

Skip Set Complete Set

Fig 6. TrackPage completed

With all the classes complete, I then played around with the app. I noticed a key issue – in the recent workouts under the DashboardPage, it was displaying the total number of sets as the total number of exercises. This was because every time the set was completed, it saved it under a duplicate entry for the list. I found the issue within `finish_and_save()` and fixed it by moving the line that appended the finished sets dictionary to its exercise in the list of exercises outside of the for each set loop, to under for each exercise loop. I finalized the app by styling, and formatting the app to fit the tone for a workout app, and even added a custom logo.

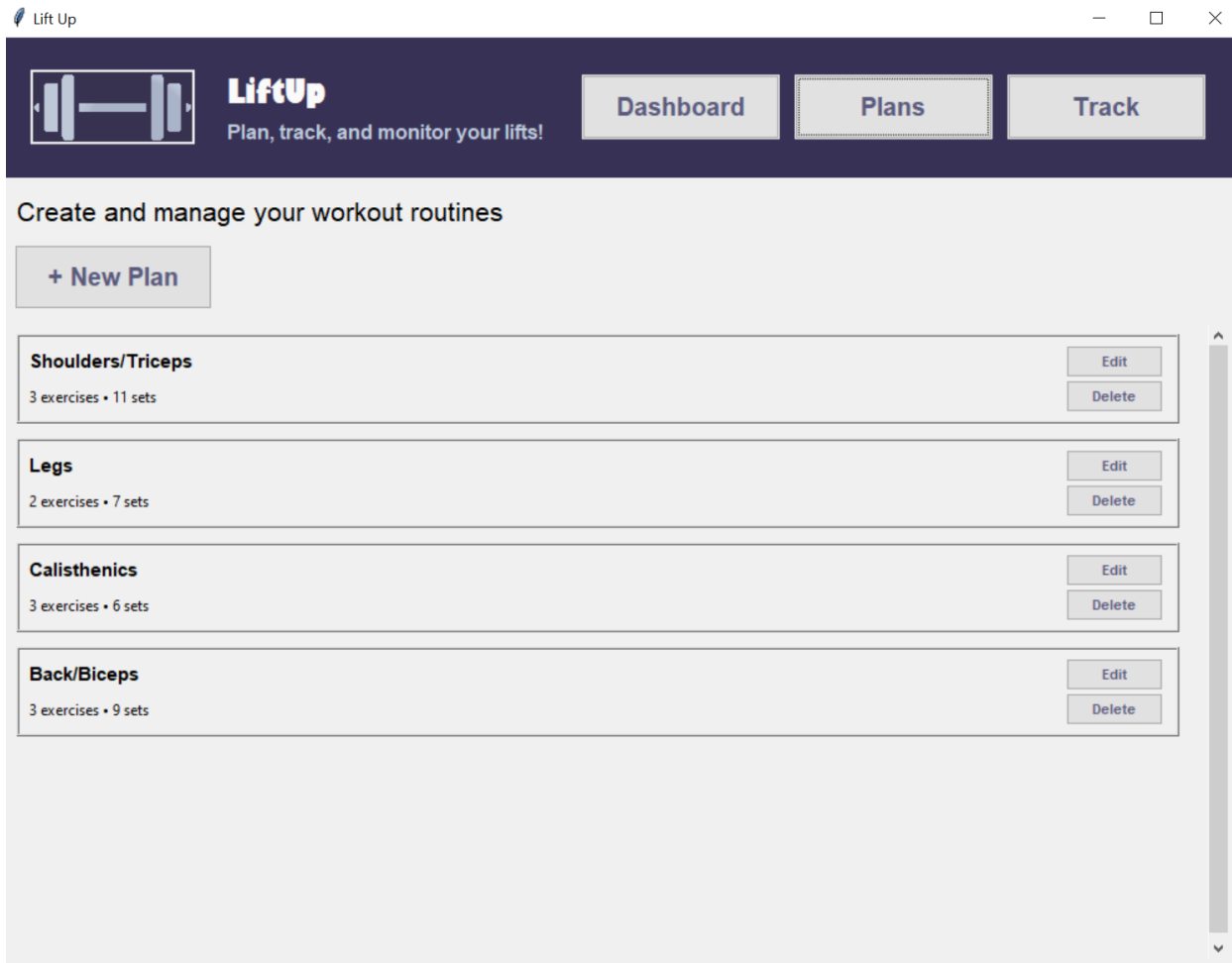


Fig 7. Stylization of the App

Results

The final result is an application designed to help users create, manage, and log their exercise routines in an organized way. The app allows users to define custom workout plans, with custom exercise names and set structures, record individual workout sessions in real time, and view their progress over time. Data is stored locally using structured json files, and read when the app starts. This ensures that plans and logs persist between sessions. The interface was built using Themed Tkinter, and it prioritizes clarity and ease of use. This allows for the application to be accessible for everyday fitness tracking.

The application has a permanent banner at the top containing the app's logo, name and subtitle, and three buttons to switch between three pages that are displayed beneath this banner. When the app is opened, by default the page displayed is the DashboardPage.

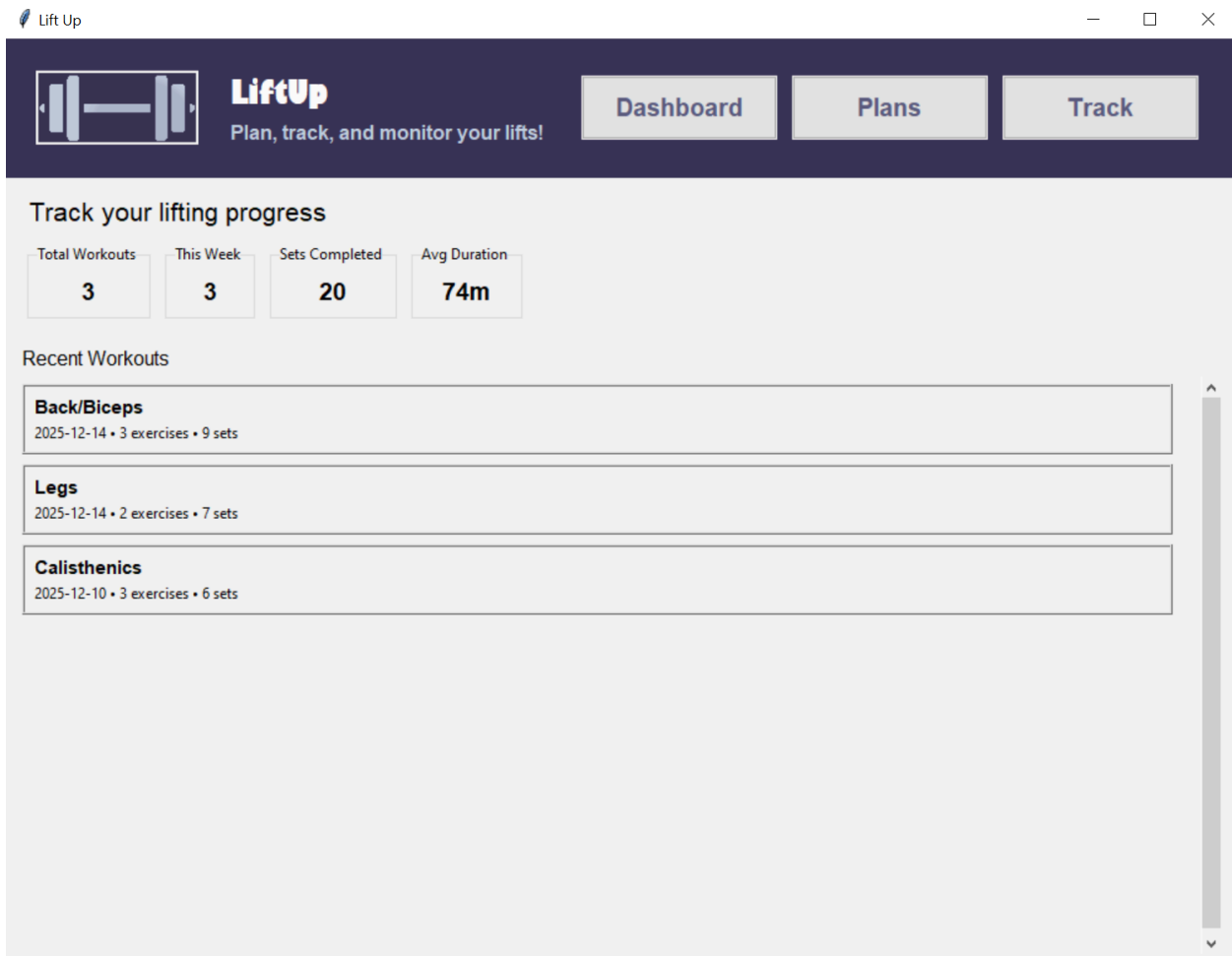


Fig 7. Application when opened

The dashboard page displays data from the WorkoutLogs data file. Towards the top of the page, it displays the total number of workouts completed, total number of workouts completed this week, the total number of sets completed across all workout sessions, and the average duration of each workout in minutes. Underneath it displays up to ten most recently completed workouts sessions with a basic summary under each workout name. The purpose of this page is to display the most relevant information from the data logs file for the user to see. This is the default page of the application and it is a passive page with no interactions from the user besides the scroll bar to view the recent workouts. The “Dashboard” button refreshes this page with the most updated information from the workoutLogs file.

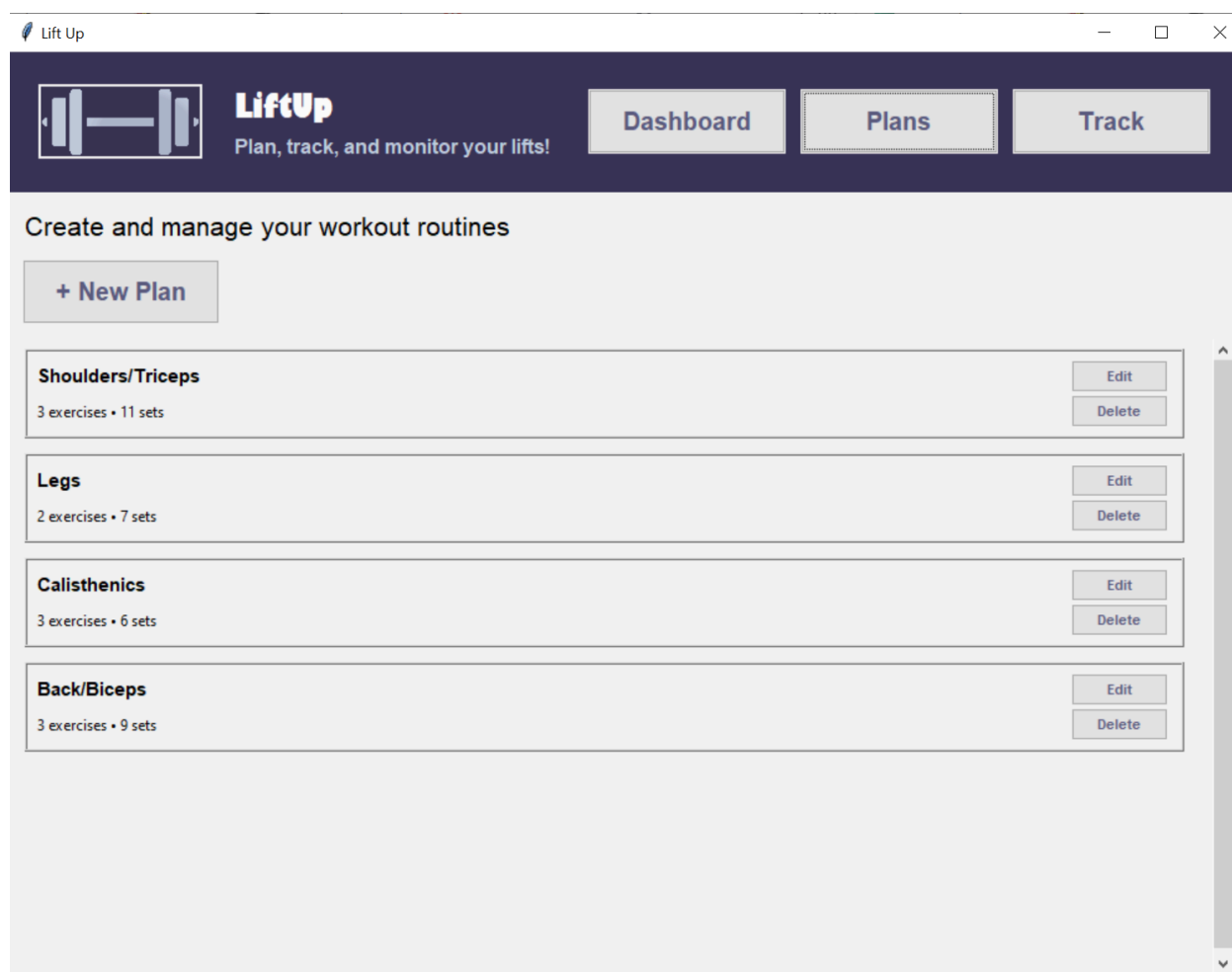


Fig 8. PlansPage showing plan's list page

The “Plans” button opens up the PlansPage. This page allows the user by default to view their list of already created plans, and to press the button “+ New Plan” to enter the page for creating a new plan. When the “Plans” button is clicked, it displays the list of already created plans. The user can use a scroll bar to go through the list if the list is long enough. The already created plans have an “Edit” and a “Delete” button that allows the user to either edit the plan – this changes the information under the already existing listing. Or delete the plan – this removes

the plan from the logs of plans. The other mode of the page is to create a new plan, or to edit a plan.

Create and manage your workout routines

+ New Plan

Create New Plan

Plan Name

Exercises

Exercise 1

Exercise Name:

Set 1 reps lbs

+ Add Set Remove Exercise

+ Add Exercise

Cancel Save Plan

Fig 9. Create a new plan in PlansPage.

When the “+ New Plan” button is clicked, it opens a page with entries for workout name, the first exercise name, and its first set details. The user needs to enter in a string for the Plan Name, and integers for the reps and weight. The user can leave the exercise name, reps, and weight as empty and it will default to “unnamed exercise” with 0 reps with 0 weight. The user can then add as many sets to an exercise as they like, and also as many exercises they like. If they want to remove an exercise, they can press “Remove Exercise” and it will remove that exercise and the set entries. The user can then either press “cancel”, and it won’t save the plan and the values entered. Or the user can press “Save Plan” and the plan will be attempted to be saved. The plan can not be saved if there is no name, or if there are non-integers entered for the reps and weight values, It will let the user know through an error message pop-up, and will go back to the plan they were creating so the user can adjust their plan properly. Once the plan is saved, the list of already saved plans is updated and displayed. Or if the user cancels, it shows the list of saved plans.

Create New Plan

Plan Name

Exercises

Exercise 1

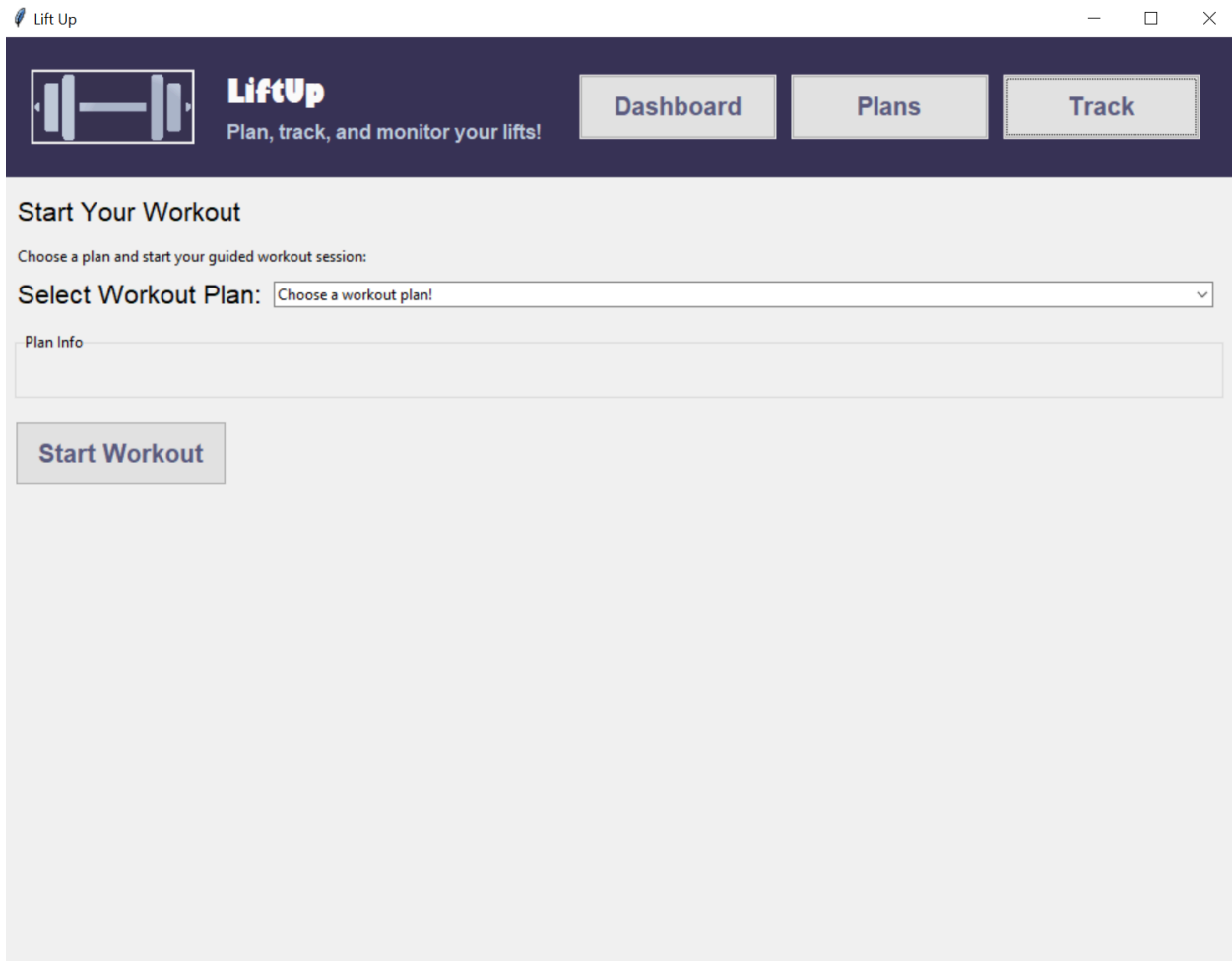
Exercise Name:

Missing name

Please enter a plan name

OK

Fig 10. Error box from missing a plan name.



The screenshot shows the 'Track' page of the 'LiftUp' application. At the top, there is a dark blue header with the 'LiftUp' logo and tagline 'Plan, track, and monitor your lifts!'. To the right of the header are three buttons: 'Dashboard', 'Plans', and 'Track'. The 'Track' button is highlighted. Below the header, the page title is 'Start Your Workout'. A sub-header reads 'Choose a plan and start your guided workout session:'. Below this is a dropdown menu labeled 'Select Workout Plan:' with the placeholder text 'Choose a workout plan!'. Underneath the dropdown is a section titled 'Plan Info' with a large empty text area. At the bottom left of the page is a button labeled 'Start Workout'.

Fig 11. TrackPage workout selection

After a workout plan is created, the user can track the plan and enter what was actually completed. This is done through the TrackPage that is accessed by pressing the “Track” button at the top of the application. This page has a droplist at the top for the user to select from the list of plans they created through the PlanPage. If there are no plans, this droplist will contain no items for the user to select, and the user can not start a workout. If the user does select a plan from the drop box, it will show a workout summary under the “Plan Info” section of the page beneath the droplist. The user can then press “Start Workout” to start tracking.

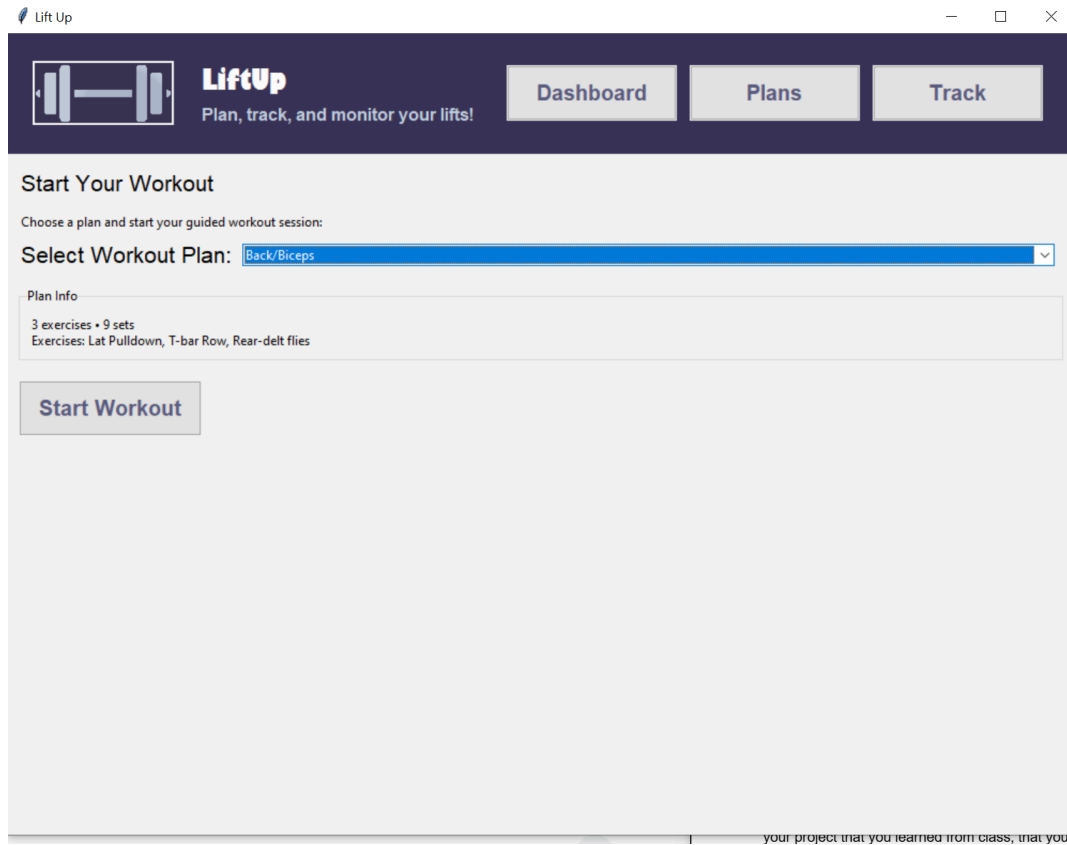


Fig 12. TrackPage with plan selected

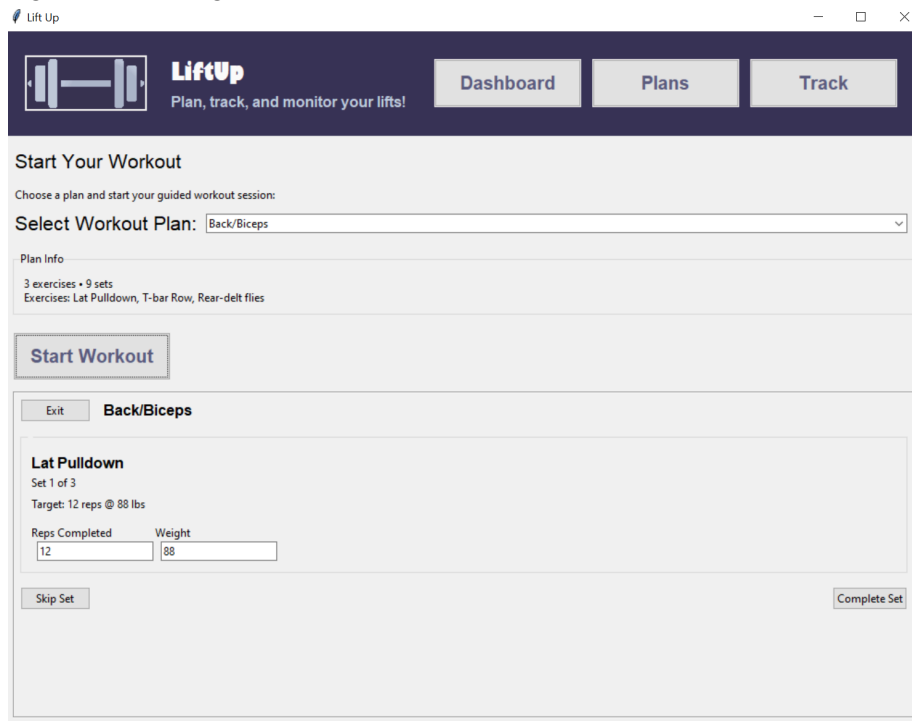


Fig 13. TrackPage with an active workout.

When “Start Workout” is pressed, it starts an active workout from the plan selected. At the top, it displays the plan name, and it has an exit button at the top left to allow the user to exit the active workout without any progress being saved. Underneath the plan name is the current exercise and set we are on. It displays the target reps and weight of the set that is pulled from the workout plan, and then has an entry to enter the amount of reps and weight actually completed by the user in this set. This can also be left unfilled by the user, as it will by default use the targets from the plan as the values. If the user wants to not complete this set, they can press “skip set” and it will go onto the next position in the workout. Otherwise, the user can press “complete set” and it will go to the next position in the workout with the information saved internally.

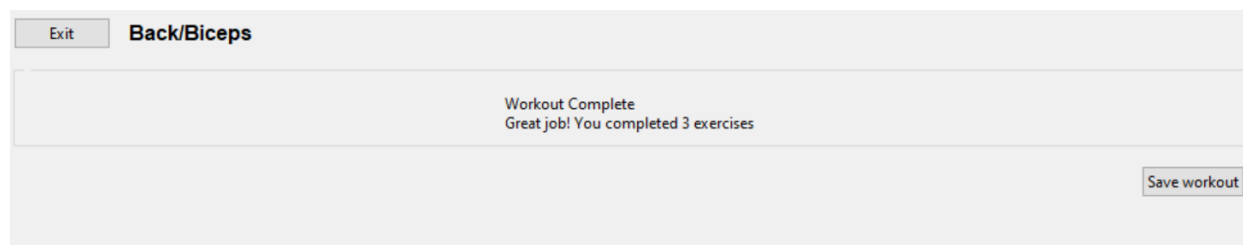


Fig 14. TrackPage when the end of workout is reached.

Once the last set and exercise is completed or skipped, it shows the user text indicating that the workout is finished, and how many exercises they completed. There is a button to the bottom right, “Save workout”, that saves the workout to the internal memory within the app, and to the json files. It then refreshes the plans page to empty the current workout section, and allow the user to select another workout plan. The user can see the completed workout at the top of the dashboard page if the dashboard page is opened.

iv. **Concepts learned:**

Through this project, I significantly developed my understanding of foundational programming concepts learned from class and how they can be applied in a real-world application. One of the most important concepts was how object-oriented programming supports code organization and scalability. Rather than writing a single main script, I learned how to decompose the program into multiple interacting classes, each responsible for a specific part of the application. My program uses an app class to run the Tkinter main loop, while also initializing, displaying, and interacting with other frame objects within the app. The frames themselves were made into classes to allow for repeated use of their own functions interacting with each other for data management and displaying.

I also gained a clearer understanding of public versus private variables and why access control matters in larger programs. While most of the Labs done in this course did require much in restricting access to variables, this project showed how limiting access helps prevent unintended side effects and makes the code easier to reason about and debug. Designing which data should be exposed to other classes and which should remain internal required deliberate planning.

Working with nested data structures strengthened my understanding of hierarchy. The workout logs contained multiple levels of hierarchy: workouts, exercises, and sets. This required careful iteration with for loops and condition checking with if statements. This reinforced how loops and conditionals can be structured to directly reflect the data structure, and taught me how to write clearer, more efficient and compact loops when dealing with complex data. I also applied this concept to the graphical user interface by organizing widgets within nested frames. Each frame is rooted within a parent frame and ultimately within the main page container, mirroring the logical structure of the application and making the interface easier to manage, update, and extend with a clear understanding of the layering of frames.

Although the course covered basic file input and output, this project expanded that knowledge by applying it to persistent application data. I learned how saving and loading data affects program flow. I prevented accidental change in data files by adding internal lists along with temporary lists from the data retrieved from the data files. Even though JSON was not covered in class, it was very easy to learn and apply in my application with its similarity to dictionaries and variable structures learnt from lectures.

Beyond course material, the project required extensive independent learning. I learned how to design a graphical user interface using Tkinter, and extended to creating a custom Tkinter object to supply an easy and adaptive scrollable frame I applied across the application. I practiced and experimented with layout management, responding to user events, and dynamically updating displayed values based on program state.

Another major learning outcome was working with time-based data. With the date-time import, I learned how to store timestamps, compute durations, compare dates, and aggregate statistics such as weekly counts and average workout duration. I applied the basic Python sorting algorithm to sort for the most recent data.

Finally this project taught me how to debug and iterate on a larger codebase. As features were added, I learned the importance of module design, clear but concise variable names, and incremental testing. Overall, the project helped bridge the gap between isolated programming exercises to building a complete, user-facing application.

v. Retrospective

If I were to redo this project or had additional time, I would focus on improving both usability and feature depth. One area I would change is how exercises are defined and executed. I would add a timed exercise type, along with a reusable countdown timer widget, to better support exercises based on duration rather than repetitions and to time rest periods between sets. This would make the application more flexible and closer to real-world workout workflows.

I would also expand the statistics and analytics provided by the dashboard. In addition to basic counts, I would include metrics such as total weight lifted per session, maximum weight lifted in a session, and long-term trends across workouts. These statistics would give users more meaningful feedback on their progress and would encourage more consistent use of the application.

Another improvement would be to redesign exercise creation. Instead of requiring users to manually enter every exercise, I would include a library of predefined exercises that users could select from, with the option to create and save custom exercises. This would reduce repetitive data entry, improve consistency across workout logs, and make the app more beginner-friendly.

From a design perspective, with more time I would also refactor parts of the code to further separate data handling, application logic, and user interface components. While the current design is functional, additional abstraction would improve maintainability and make it easier to add new features in the future.

Overall, these changes would make the application more effective, scalable, and aligned with real fitness tracking tools, while reinforcing the importance of planning for extensibility early in the development process.